

DEEPL - TP 7

Convolutional Networks (1D)

Nicolas Baskiotis - Stéphane Rivaud - Benjamin Piwowarski - Laure Soulier

2025-2026

Introduction (brief, see lecture)

A convolutional neural network (CNN for "Convolutional Neural Network") is a type of architecture primarily used for *sequence classification* (1D), images (2D) or, in general, data presenting symmetries. In the most standard cases, these symmetries mean that the way of studying an individual is invariant with respect *to a translation*. For example, detecting a mouth in an image does not depend on the position of that mouth in the image; for text, detecting that we are talking about a red animal also does not depend on the position of this excerpt in the text.

1 Sentiment Detection

In the remainder of this practical session, we will conduct experiments with 1D CNNs to classify sequences, but the operating principle is general.

Question 1

You will use variable segmentation with *sentencepiece* (use the provided script `tp8_preprocess.py` for this). Start the process now by specifying the desired vocabulary size (it may take a while).

In 1D (figure 1), convolution can be seen as a linear transformation that operates on `kernel_size` elements of the sequence (3 in the figure) repeated by moving `stride` elements each time (1 in the figure).

Each linear transformation transforms a vector of size `kernel_size × in_channels` into a vector of size `out_channels` where `in_channels` is the dimension of the input data, and `out_channels` is the output dimension. This output dimension can be viewed as the number of filters on sequences of size `kernel_size`. In this practical session, you will use the `torch.nn.Conv1d` module from PYTORCH which efficiently implements these convolutions.

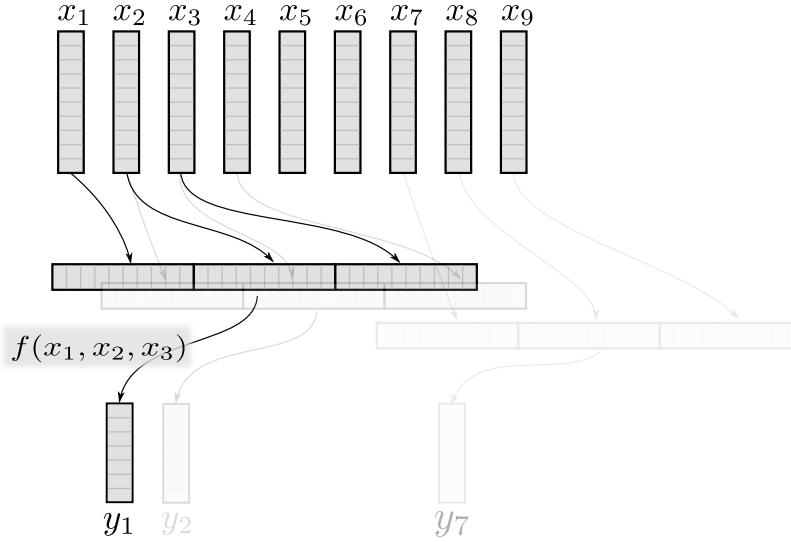


FIGURE 1 – A 1D convolution (with $f(x_1, x_2, x_3) = (x_1 \oplus x_2 \oplus x_3)A + b$) **or** a pooling operator (e.g. for max-pooling : $f(x_1, x_2, x_3) = (\max_{i=1\dots 3} x_{i1}, \dots, \max_{i=1\dots 3} x_{id})$)

Another common layer in CNNs corresponds to a *pooling* operator. Its operation is similar to the convolution operator in every way except for the function used for transformation. For pooling, it involves an aggregation performed at the level of each component of the input vectors. For example, *max-pooling* returns the maximum (`torch.nn.MaxPool1d`).

The interaction between the convolution operator and max-pooling is as follows :

1. Convolution allows detection of a set of patterns (for example, "a good movie", "very good movie" or "a deplorable movie") on short sequences (here 3 words) ;
2. *Max-pooling* allows summarizing the information captured by the filters over a larger subsequence. For example, a max-pooling of size 5 and stride 2 will summarize the detections for subsequences $(y_1, \dots, y_5)$, then $(y_3, \dots, y_7)$, etc.

It is then possible to repeat these operations a certain number of times, in order to detect patterns of increasing size and complexity. For example, we can detect "I really"/"I enormously"/"not much" and "this movie"/"this actor" (4 filters) in the first convolution layer. The second convolution layer will combine the information to detect a judgment about a movie or an actor.

The last layer is generally a global maximum (on each output component), followed by a linear classifier.

The data used comes from Sentiment140. The classes are 0 = negative, 2 = neutral, 4 = positive (converted to 0, 1 and 2 in the preprocessed data).

Question 2

After inspecting the type of data, you will propose convolutional architectures that you will compare in terms of classification accuracy ; you will take care to measure the performance of a trivial algorithm that systematically returns the majority class and report the performance relative to this model.

2 Study of CNN Behavior

In order to study what the CNN does, we will focus on the last layer before the global maximum ; more specifically, we will search for the subsequences (in the *train* set) that most activate each output filter.

Example To do this, we must first determine which position (in the text) corresponds to each output. For example, if we consider a convolution with a kernel size of 3 and a stride of 1, then the 1st output will correspond to the text between positions 1 and 3, the 2nd to that between positions 2 and 4, etc. If we add max-pooling (kernel size 2, stride 2), then the 1st output corresponds to the text between positions 1 and 4, the 2nd to positions 3 and 6, etc.

We must now generalize. For this, we will consider that the i th operation (convolution/-pooling) is defined only by the kernel size w_i (*kernel width*) and the displacement s_i (*stride*). We are interested in two values that characterize the set of transformations up to operation $i - 1$:

- The length of the inputs corresponding to an output W_{i-1} .
- The displacement S_{i-1} in the inputs corresponding to a unit displacement in the outputs.

Question 3

Give the recurrence formula that allows, given W_i , S_i , to determine W_{i+1} and S_{i+1} knowing w_{i+1} and s_{i+1} .

Question 4

Let $(y_1, \dots, y_L)$ be the output of the CNN. Once this calculation is done, give the formula that, given the position j of the output y_j , allows determining the corresponding indices in the input sequence.

Question 5

Finally, go through the data from the *train* set, and find the subsequences that most activate each output feature.